

AIM- To configure a GPIO pin of STM32F446RE as digital output and verify LED blinking operation using software delay routines.

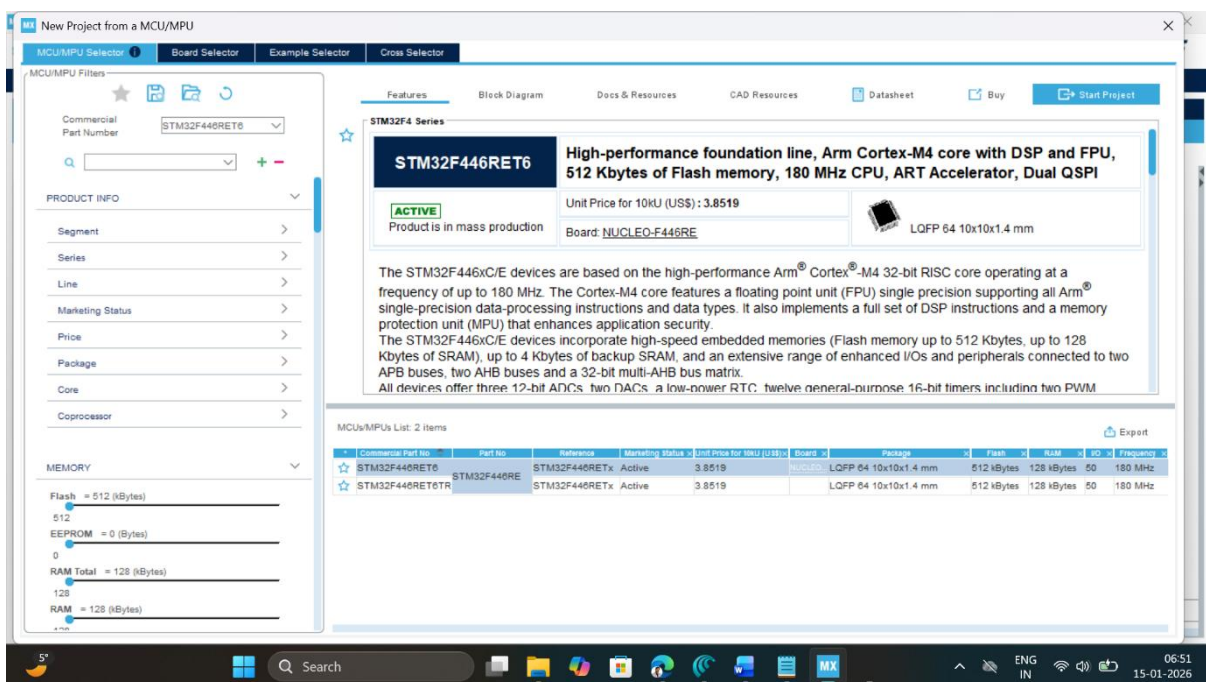
APPARATUS- STM32 Nucleo-F446RE development board, USB Type-A to Mini-B cable, STM32 CUBE IDE, STM32 CubeMX.

THEORY

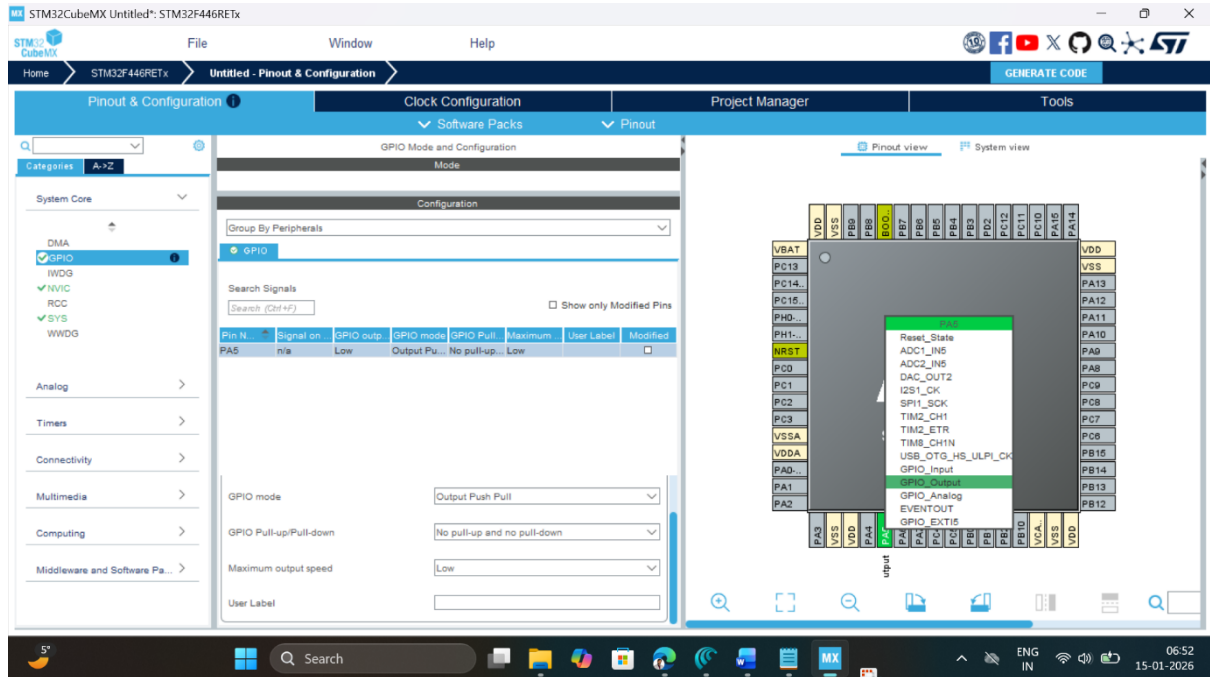
STM32F446RE provides multiple general-purpose I/O (GPIO) ports (GPIOA–GPIOH) that can be configured as input, output, alternate function, or analog through GPIO control registers or HAL. On the Nucleo-F446RE board, the green user LED **LD2** is connected to Arduino D13, which corresponds to microcontroller pin **PA5** on most board revisions. When PA5 is configured as a **push-pull output**, the pin can be driven to logic high (near 3.3 V) or logic low (0 V), which in turn turns the LED on and off through an on-board current-limiting. Using STM32CubeIDE and HAL, functions like HAL_GPIO_TogglePin() and HAL_Delay() can be used inside an infinite loop to generate a periodic LED blink without directly programming registers.

PROCEDURE

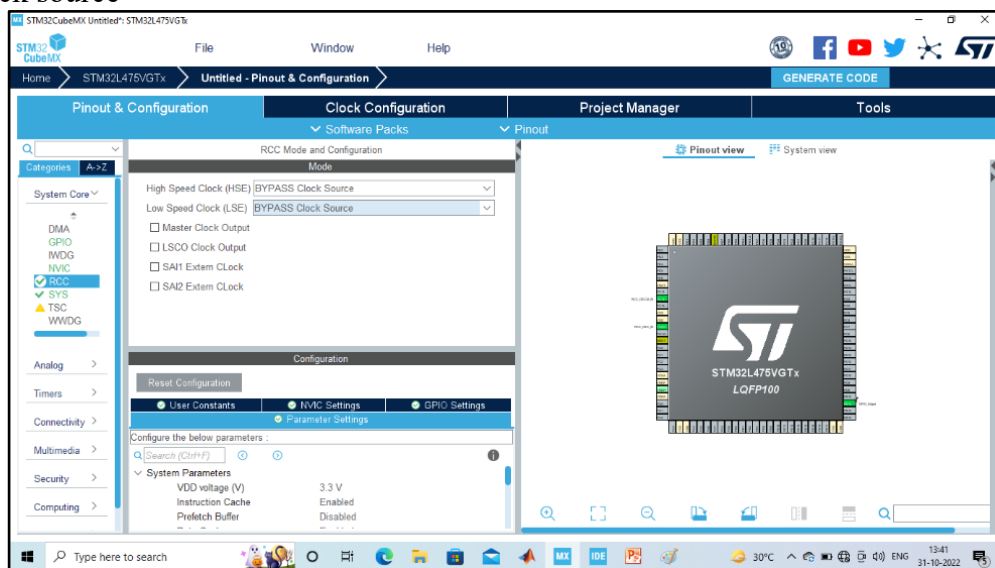
1. Open CubeMX and Click on ACCESS TO MCU SELECTOR.



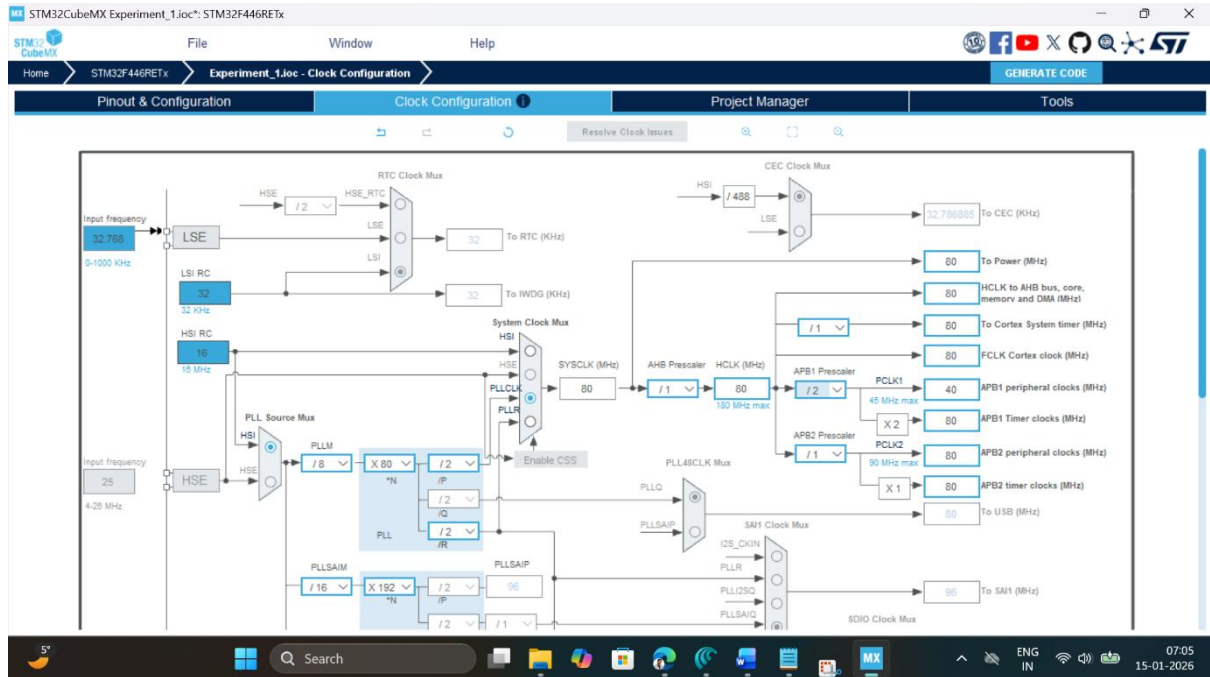
2. Write and select the microcontroller as you want to use and then click start the project. Select commercial part number STM32L446RE.
3. In system Core select GPIO and right click on pin PA5 to make it as GPIO output pin as shown below as LED (Inbuilt LED) relates to it in the board.



4. **Clock Selection:** Select the internal clock by clicking RCC and selecting BYPASS clock source



5. **Clock Configuration:** Configure clock as shown below to get maximum internal clock frequency.



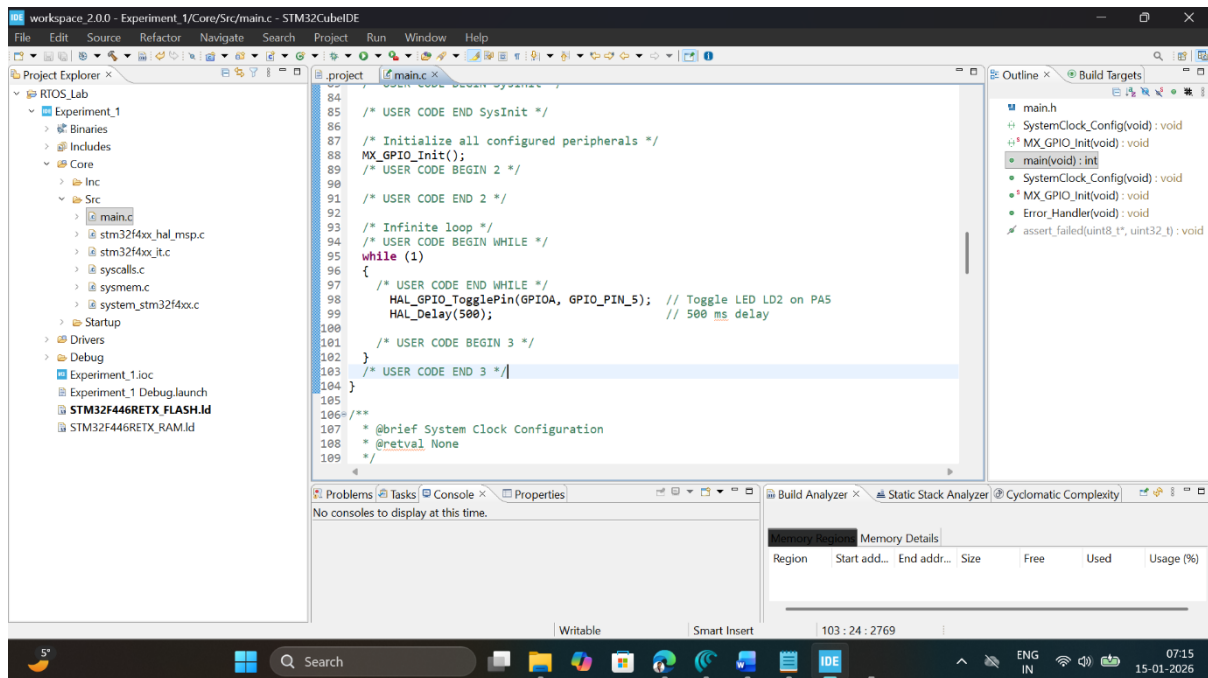
6. Write Project name and select IDE.

The screenshot displays the STM32CubeMX Project Manager interface. The 'Project' tab is selected, showing the following settings:

- Project Name:** Experiment_1
- Project Location:** C:\Users\dell\Documents\RTOS_Lab
- Application Structure:** Advanced
- Toolchain / IDE:** STM32CubeIDE
- Linker Settings:**
 - Minimum Heap Size: 0x200
 - Minimum Stack Size: 0x400
- Thread-safe Settings:**
 - Enable multi-threaded support: ☐
 - Thread-safe Locking Strategy: Default - Mapping suitable strategy depending on RTOS selection.
- Mcu and Firmware Package:**
 - Mcu Reference: STM32F446RETx
 - Firmware Package Name and Version: STM32Cube_FW_F4_V1.28.3
 - Use latest available version: ☒
 - Use Default Firmware Location: ☒
 - Firmware Relative Path: C:\Users\dell\STM32Cube\Repository\STM32Cube_FW_F4_V1.28.3

- Click on generate code.
- Click on open project.
- Click 'OK' on the generated popup showing "successfully imported the project on the workspace" and the project will be successfully added to the cube IDE.
- Open the main source code on Cube IDE by clicking on **main.c**
- To generate bin file and hex file, right click on project and click properties.
- Expand C/C++ Build, click on setting, click on MCU post build outputs and select convert to binary file as well as convert to hex file and click "Apply and Close".
- After configuration write only below two instructions in the user while loop and compile the program and ensure there will be zero error after compiling.

```
HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
HAL_Delay(500);
```



14. To blink the connected LED on the board, connect the board, click on build, click 'ok' and click 'switch' on popups.
15. Now you are in debugging mode, click 'Resume' to execute the program.

OBSERVATION: Change the delay value (e.g., 200 ms, 1000 ms), re-build, re-run, and note the change in blink speed and fill the observation table.

S. No.	GPIO pin used	Mode / Type / Pull	Delay value (ms)	Approx. blink period (s)	Comment (slow/fast/visible)
1	PA5	Output, PP, No PU/PD	100	0.2	FAST
1	PA5	Output, PP, No PU/PD	200	0.4	FAST
2	PA5	Output, PP, No PU/PD	500	1	FAST
3	PA5	Output, PP, No PU/PD	1000	2	SLOW

RESULT

The GPIO pin PA5 of STM32F446RE on the Nucleo-F446RE board was configured as a digital output in STM32CubeIDE, and the on-board user LED LD2 was successfully made to blink using software delay routines implemented with HAL functions

Write down reflection summary (min 200 words) on your understanding of GPIO configuration parameters of STM32 MCU's?

Through this experiment, I gained a clear and practical understanding of GPIO (General Purpose Input/Output) configuration parameters in STM32 microcontrollers. GPIO pins are one of the most fundamental resources of an MCU, as they act as the interface between the microcontroller and external hardware such as LEDs, switches, sensors, and communication modules. STM32 MCUs provide highly flexible GPIO functionality, allowing each pin to be configured as input, output, alternate function, or analog mode depending on application requirements.

I learned that selecting the correct GPIO mode is crucial for proper system behavior. For example, configuring a pin as a push-pull output enables it to actively drive both logic high and logic low levels, making it ideal for driving LEDs. Parameters such as output speed, pull-up/pull-down resistors, and alternate function mapping further refine how the pin interacts electrically and logically with external components. Misconfiguration of these parameters can lead to unstable signals, excessive power consumption, or even hardware damage.

Using STM32CubeMX and STM32CubeIDE simplified GPIO configuration by abstracting low-level register programming and allowing visual pin assignment. HAL library functions like `HAL_GPIO_TogglePin()` and `HAL_Delay()` demonstrated how GPIO pins can be controlled efficiently through software, improving code readability and development speed. Overall, this experiment helped bridge the gap between theoretical knowledge and real hardware implementation, strengthening my confidence in configuring and using GPIO peripherals in STM32-based embedded systems.

